INTELLIGENT CODE EXPLAINER SYSTEM

Project Report

Bachelor of
Technology In

Computer Science and Engineering
School of Engineering and Sciences

Submitted By:

A.Javeed	AP23110010947
Y.Lohith sai	AP23110010951
D.Jathin rishikeshav	AP23110010943
D.Hanumanth sai kumar	AP23110010954

Under the Guidance of

Mr.Ajay Dilip Kumar
Lecturer, Department
of CSE SRM
University–AP

Neerukonda, Mangalagiri, Guntur
Andhra Pradesh – 522 240

[April, 2026]

TABLE OF CONTENTS

Abstract 3

1. Introduction 4
2. Problem Statement 5
3. System Design 6
   3.1 Architecture Overview 6
   3.2 Module Design 7
   3.3 Algorithm Design 7
      3.3.1 Diagnostic Parsing 7
      3.3.2 Machine Learning Classification 8
      3.3.3 Fallback Mechanism 8
      3.3.4 Execution Flow 8
4. Implementation 9
   4.1 Execution Flow 9
   4.1.1 Flow Representation 10
   4.1.2 Performance Optimization 11
   4.1.3 Time Complexity 11
5. Technologies Used 11
6. Outputs 13
7. Test Cases 14
8. Conclusion 15
9. Limitations 15

ABSTRACT

The **C Error Explainer** is an AI-assisted debugging system designed to convert complex compiler error messages into clear, human-readable explanations. The system accepts C source code, compiles it using GCC, and analyzes the resulting diagnostics to generate structured explanations including causes and suggested fixes.

Unlike cloud-based tools, the system operates entirely offline using a locally hosted backend built with FastAPI. It employs a hybrid approach combining a **machine learning model (TF-IDF + Logistic Regression)** with a **rule-based fallback mechanism** to ensure reliability. Additionally, successful programs are executed, and runtime outputs are displayed.

This project demonstrates an effective integration of **compiler design, machine learning, and web technologies** to create an intelligent debugging assistant that enhances both productivity and learning.

1. INTRODUCTION

Programming in C is both powerful and widely used, but it presents significant challenges, particularly for beginners. One of the major difficulties lies in understanding compiler error messages generated by tools such as GCC. These messages are typically technical, concise, and lack clear guidance, making the debugging process complex and time-consuming. As a result, users often struggle to identify the root cause of errors and apply appropriate fixes, which slows down learning and development.

The **C Error Explainer** is designed to address this problem by converting complex compiler diagnostics into simple, human-readable explanations. It not only interprets errors but also identifies their underlying causes and suggests possible solutions, thereby improving the overall debugging experience. The system is implemented as a locally hosted web-based application, ensuring that all processing is performed on the user's machine. This eliminates the need for internet connectivity, enhances data security, and provides faster response times.

A key strength of the system is its hybrid intelligence approach, which combines machine learning with rule-based techniques. The machine learning model enables intelligent classification and understanding of compiler errors, while a rule-based fallback mechanism ensures consistent performance even if the model is unavailable or fails. This combination guarantees reliability, efficiency, and continuous system functionality, making the C Error Explainer a robust and user-friendly solution for code debugging.

2. PROBLEM STATEMENT

Traditional compilers such as GCC generate diagnostic messages that are often low-level, technical, and difficult for users—especially beginners—to understand. While these messages accurately indicate errors, they do not clearly explain why the error occurred or provide guidance on how to fix it. As a result, users spend significant time interpreting compiler outputs, which slows down development and creates a steep learning curve.

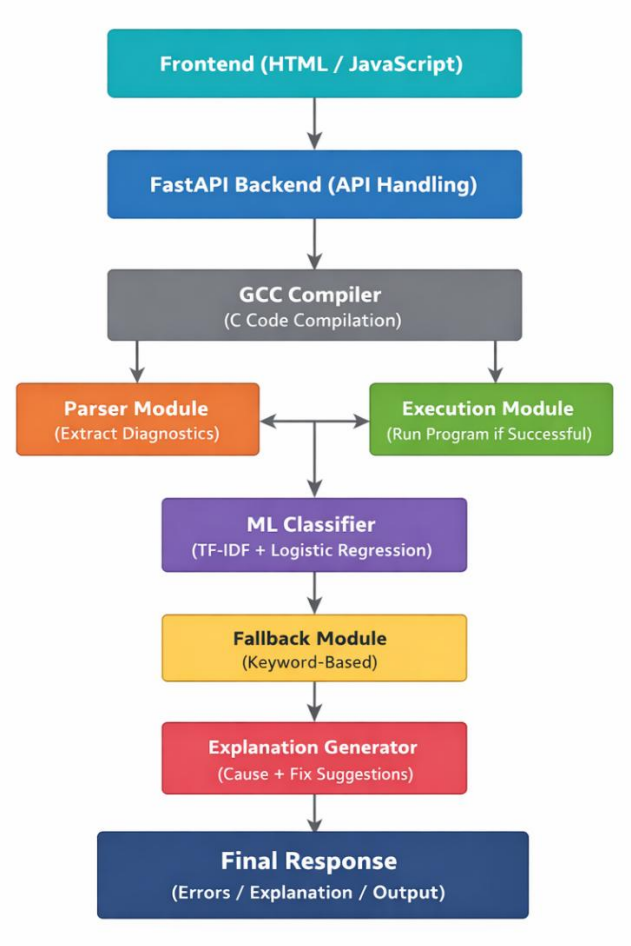
The problem is further complicated by the complexity of GCC diagnostic formats, which vary depending on the type of error and context. Handling multiple categories of errors, including syntax, semantic, and runtime issues, requires a structured and intelligent approach. Additionally, the system must process these errors in real time to provide immediate feedback, ensuring a smooth user experience. Another important constraint is the requirement for offline functionality, meaning the system must operate without relying on external APIs or internet connectivity.

Therefore, the objective of this project is to design and develop an intelligent system that can automatically interpret compiler errors, convert them into meaningful and human-readable explanations, and provide guidance for resolving them. The system should be efficient enough to operate in real time while maintaining accuracy and reliability, and it must function entirely offline to ensure accessibility, security, and independence from external services.

3. SYSTEM DESIGN

3.1 Architecture Overview

The system follows a **pipeline-based modular architecture**:



3.1.1 Architecture Explanation

The C Error Explainer follows a modular pipeline architecture where each component performs a specific task in sequence. The process begins at the frontend, where the user enters C code using a simple HTML/JavaScript interface. This code is sent to the FastAPI backend, which acts as the central controller and manages all further operations such as compilation, analysis, and response generation.

The backend forwards the code to the GCC compiler, which attempts to compile it. If errors occur, the compiler generates diagnostic messages. These messages are then processed by the parser module, which extracts structured information such as line number, error type, and

description. This structured data makes it easier for the system to analyze and classify errors efficiently.

Next, the parsed error message is passed to the machine learning classifier, which uses TF-IDF and Logistic Regression to categorize the error into predefined types. If the ML model is unavailable or fails, a fallback mechanism based on keyword matching is used to ensure reliability. Once the error category is identified, the explanation generator produces a simple, human-readable explanation along with possible causes and fixes.

If the code compiles successfully, the execution module runs the program and captures its output. Finally, all results—including errors, explanations, or execution output—are returned to the frontend as a structured response. This architecture ensures the system is efficient, reliable, and helpful for both debugging and learning.

3.2 Module Design

Component	**Purpose**	**Reason**
FastAPI Backend	Handles API requests	High performance
GCC Compiler	Compiles C code	Standard tool
Parser Module	Extracts diagnostics	Structured processing
ML Module	Classifies errors	Intelligent output
Frontend (JS)	User interaction	Lightweight

3.3 ALGORITHM DESIGN

3.3.1 Diagnostic Parsing

The system employs a structured approach to extract meaningful information from compiler outputs. Since GCC generates error messages in a specific textual format, a regular expression-based parsing technique is used to identify and extract relevant components from each diagnostic line. These components include the file name, line number, column position, and

the actual error message. This structured extraction enables the system to convert unorganized compiler output into a standardized format that can be further processed by subsequent modules.

3.3.2 Machine Learning Classification

To intelligently interpret compiler errors, the system utilizes a machine learning-based classification mechanism. The input to this module is the diagnostic message obtained from the parsing stage, and the output is a predefined error category that represents the type of issue encountered.

The classification model is built using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization to convert textual error messages into numerical feature representations. These features are then processed by a Logistic Regression model, which predicts the most appropriate error category. This approach enables the system to generalize across different variations of similar error messages and provide consistent interpretations.

3.3.3 Fallback Mechanism

To ensure robustness and uninterrupted functionality, the system incorporates a fallback mechanism in cases where the machine learning model is unavailable or fails to produce a prediction. In such scenarios, a rule-based keyword matching technique is employed. This method compares the diagnostic message with predefined patterns, and the longest matching keyword is used to determine the error category. This ensures that the system continues to provide explanations even in the absence of the trained model, thereby enhancing reliability.

3.3.4 Execution Flow

The overall working of the system follows a sequential pipeline. Initially, the user submits C source code through the interface, which is then compiled using the GCC compiler. The resulting output, including any errors or warnings, is parsed to extract structured diagnostic information. These diagnostics are then passed to the classification module, where each error is categorized using either the machine learning model or the fallback mechanism. Based on

the identified category, appropriate explanations, causes, and suggested fixes are generated. Finally, if the code compiles successfully without errors, the system proceeds to execute the program and returns the runtime output along with the analysis results.

4. IMPLEMENTATION

4.1 Execution Flow

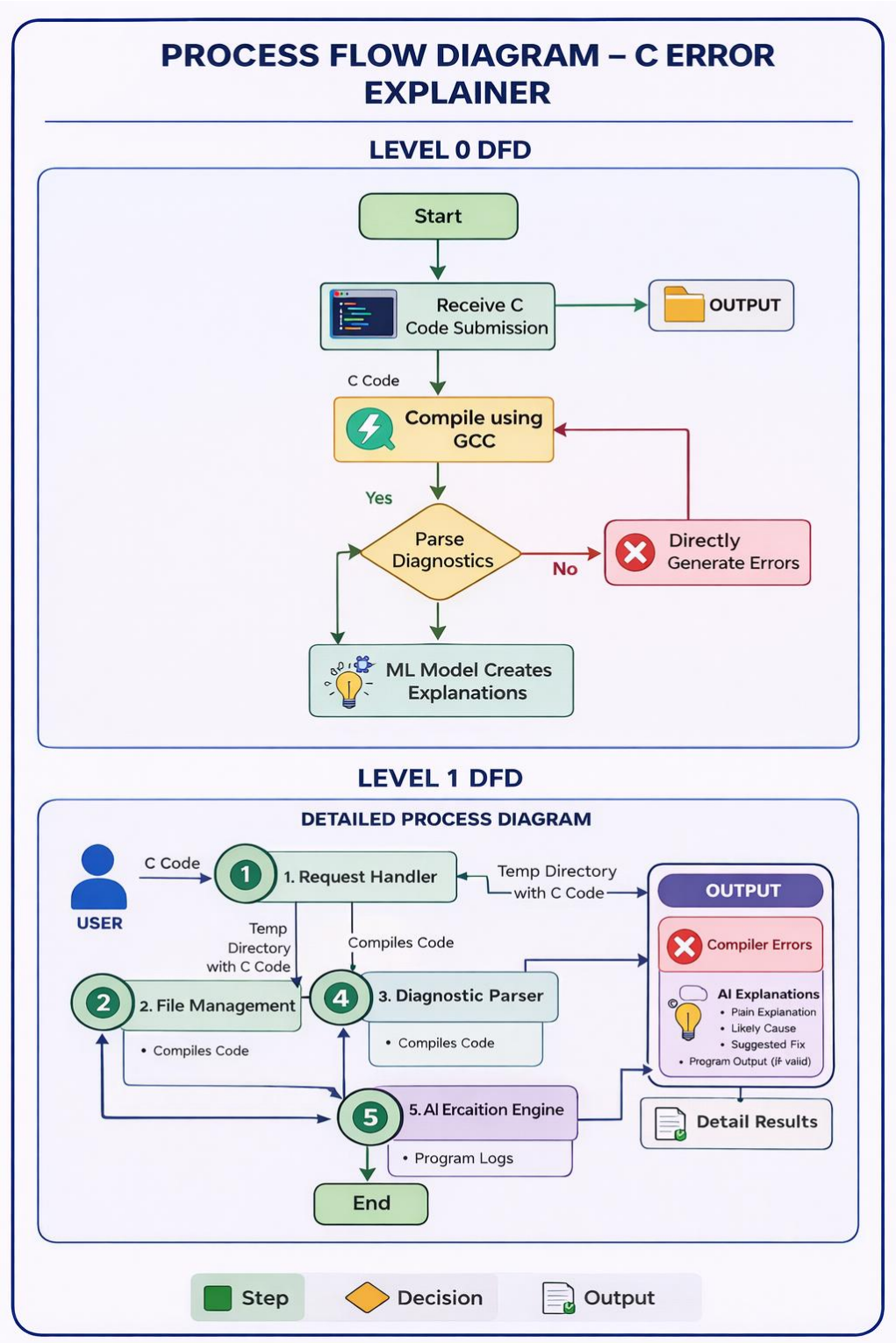
The execution flow of the C Error Explainer begins when the user submits C source code through the frontend interface. This code is sent to the FastAPI backend, where it is temporarily stored as a file. The backend then invokes the GCC compiler to compile the program. Based on the compilation result, the system follows two possible paths.

If the compilation fails, the compiler generates diagnostic messages, which are passed to the parser module. The parser extracts relevant details such as error type and location, and this information is then forwarded to the machine learning classifier. The classifier categorizes the error, and if it is unable to do so, the fallback mechanism uses keyword matching to identify the error type. The explanation generator then produces a clear, human-readable explanation along with possible causes and suggested fixes.

If the compilation is successful, the system bypasses the error analysis stages and moves to the execution module. Here, the compiled program is executed, and the runtime output is captured. This allows the user to see the actual result of their program in addition to verifying that it works correctly.

Finally, the backend collects all outputs—whether error explanations or execution results—and sends them back to the frontend in a structured format. The frontend displays this information to the user, completing the entire process from code input to debugging or execution output in a seamless manner.

4.1.1 Flow Representation



4.1.2 Performance Optimization

Technique	Purpose	Result
Timeout Control	Prevent infinite loops	Safe execution
Lazy Model Loading	Reduce startup time	Faster response
Fallback Logic	Ensure reliability	No system failure

4.1.3 Time Complexity

Component	Complexity
Parsing	O(n)
ML Prediction	O(n)
Compilation	Depends on GCC

5. TECHNOLOGIES USED

- **Backend: FastAPI**

  → Used to build the server-side of the application

  → Handles API requests and responses between frontend and system

  → Controls execution flow (compilation, parsing, ML, output)

  → Chosen for high performance and easy integration with Python

- **Frontend: HTML, JavaScript**

  → Provides user interface for entering C code

  → Sends code to backend and displays results

  → Shows compiler errors, explanations, and execution output

  → Lightweight and easy to implement for web interaction

- **Compiler: GCC (GNU Compiler Collection)**

  → Compiles the C source code into executable form

→ Generates error and warning messages if code is incorrect

→ Acts as the core component for detecting issues in code

→ Widely used and reliable standard C compiler

- **Machine Learning: Scikit-learn**

  → Used to implement ML-based error classification

  → Provides TF-IDF for text processing of error messages

  → Helps convert textual errors into numerical features

  → Simple and efficient library for building ML models

- **Model: Logistic Regression**

  → Classifies compiler errors into predefined categories

  → Works on processed TF-IDF features

  → Lightweight and fast, suitable for real-time prediction

  → Helps generate accurate and relevant explanations

- **Server: Uvicorn**

  → Runs the FastAPI application

  → Handles incoming requests and sends responses quickly

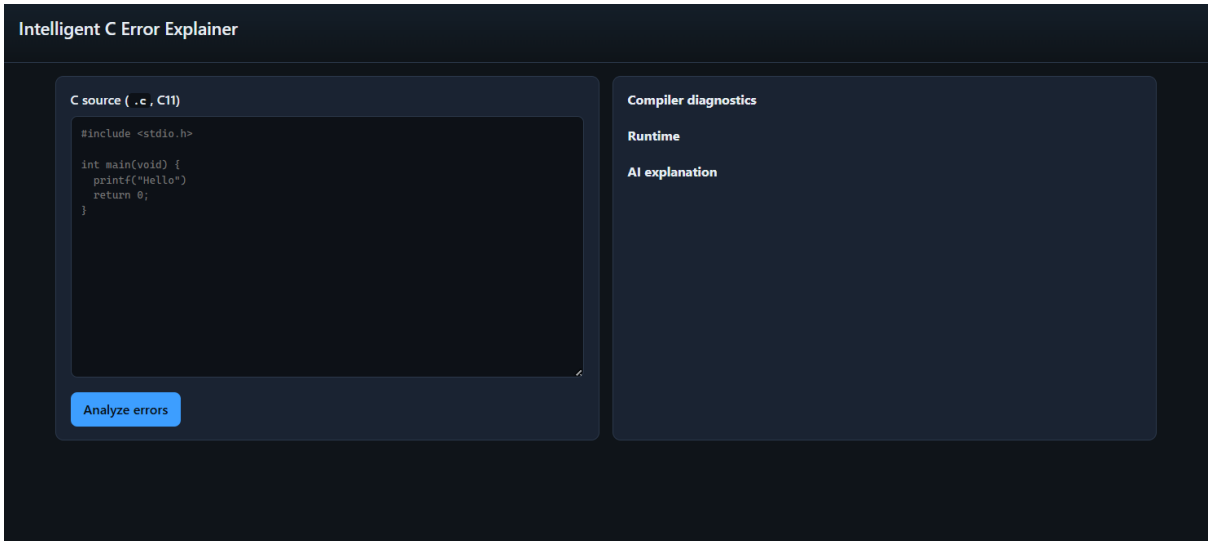
  → Supports asynchronous processing for better performance

  → Ensures smooth communication between frontend and backend

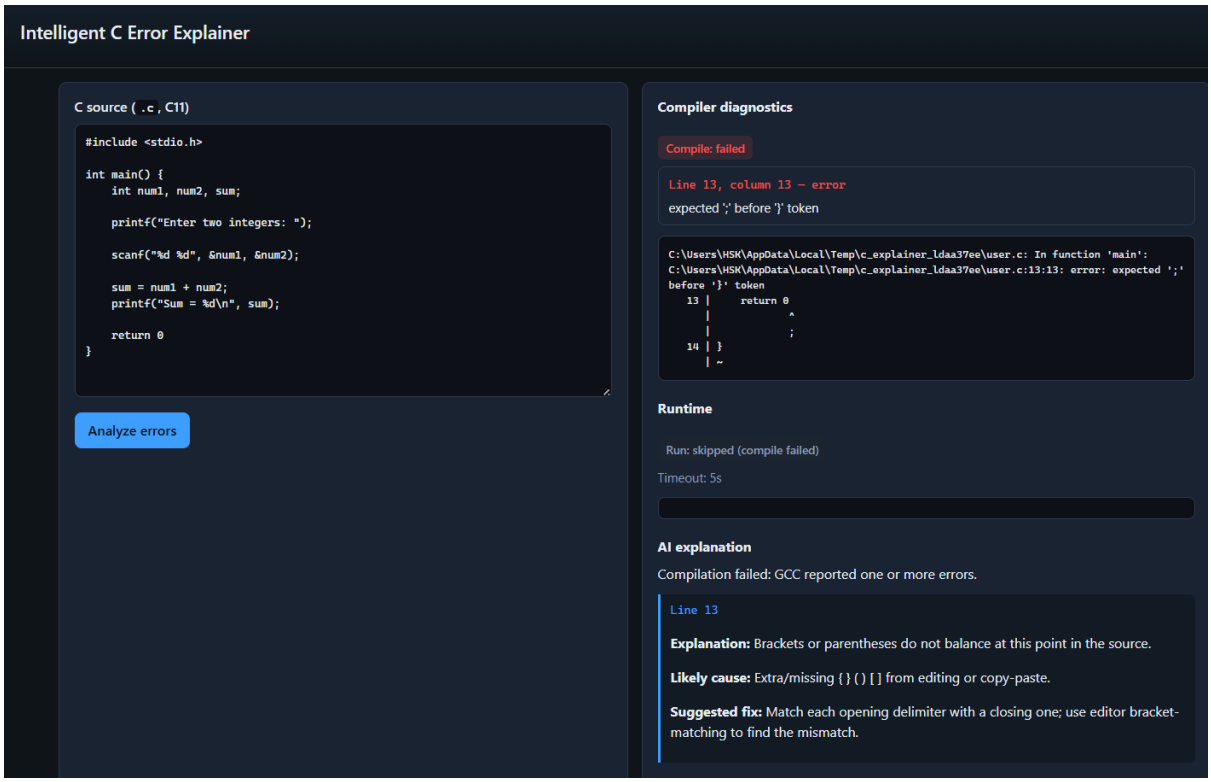
6. OUTPUTS

The system provides:

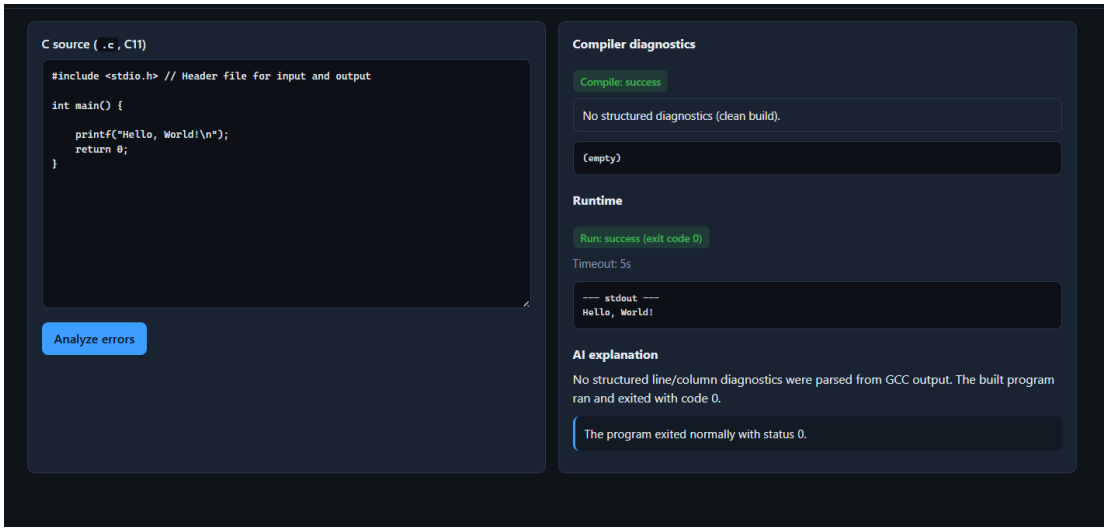
1. **Code Input Interface**



2. **AI Explanation**



3. **Program Execution Output**



7. TEST CASES

Test ID	Scenario	Expected Result	Status
TC1	Valid Code	Compiles & runs	Pass
TC2	Syntax Error	Explanation generated	Pass
TC3	Missing Library	Proper suggestion	Pass
TC4	Runtime Error	Runtime message	Pass
TC5	ML Failure	Fallback works	Pass

8. CONCLUSION

The **C Error Explainer** successfully converts complex compiler errors into understandable explanations, improving both debugging efficiency and learning.

Key Achievements

- Offline intelligent debugging system
- Hybrid ML + rule-based approach
- Complete development cycle support

Future Enhancements

- Larger training datasets
- Secure sandbox execution
- Advanced deep learning models

This project demonstrates strong integration of:

- Compiler systems
- Machine learning
- Web technologies

9. LIMITATIONS

The C Error Explainer, while effective, has certain limitations that can be addressed in future improvements. One of the primary limitations is its dependence on the training data used for the machine learning model. Since the Logistic Regression model is trained on a limited set of compiler error messages, it may not accurately classify rare or complex errors, leading to less precise explanations.

Another limitation is that the system mainly focuses on **syntactic and compilation errors** and has limited capability in handling deeper logical or semantic errors in programs. For example, issues related to incorrect algorithms or inefficient logic cannot be effectively explained by the current system.

The system is also restricted to the **C programming language** and relies specifically on GCC compiler outputs. This makes it less flexible, as it cannot directly support other programming languages or compilers without additional modifications. Furthermore, any changes in the GCC diagnostic format may affect the parsing and classification process.

Although a fallback mechanism is implemented for reliability, the **keyword-based approach may not always produce accurate results**, especially for ambiguous or multi-context errors. Additionally, the system executes user programs without advanced sandboxing, which may pose security risks when handling unsafe or malicious code.